Pontifícia Universidade Católica do Paraná – PUCPR
Escola Politécnica – Campus Sede – Curitiba/PR
Curso Bacharelado em Ciência da Computação
Disciplina de **Complexidade de Algoritmos**
Abílio Pedro Alcântara Mota Batista

PjBL – TRABALHO FINAL: DEDUPLICAÇÃO DE LEITURAS DE SENSORES

1 – Recomendação final por cliente

Para a **Embarca Ltda.**, recomendamos a **ordenação in-place com heapsort seguida de varredura de vizinhos** (estratégia 2 do catálogo). Ela usa **71 bytes** de memória extra, sendo 0 de heap, muito abaixo do limite de 256 bytes, e processa 10^7 leituras em 1,951 s.

Para o **Datacenter S.A.**, recomendamos o **bitmap de 65 536 bits** sobre o domínio conhecido (estratégia 5). Ele processa 10^7 leituras em **12,34 ms**, cerca de 162 vezes abaixo do limite de 2 s, usando 8 192 bytes de bitmap.

Cliente	Estratégia recomendada	Resultado medido ($n = 10^7$)	Restrição
Embarca Ltda.	2. Ordenação in-place (heapsort) + varredura	71 bytes extras (heap 0), 1,951 s	≤ 256 bytes: **cumpre**
Datacenter S.A.	5. Bitmap de 65 536 bits	12,34 ms, 8 192 bytes de bitmap	< 2 s: **cumpre**

2 – Por que essas duas estratégias

Elas ficam nos dois extremos do compromisso entre tempo e espaço, e cada uma cumpre a restrição do seu cliente e quebra a do outro. Por isso não faz sentido trocá-las:

- O bitmap usa 8 192 bytes, **32 vezes** o limite da Embarca. O heapsort leva 1,951 s para 10^7 leituras, quase no limite do Datacenter (uma das 5 rodadas passou de 2 s: 2,079 s), e é **158 vezes mais lento** que o bitmap.
- **Alternativas descartadas para a Embarca:** só a força bruta e a ordenação cabem em 256 bytes. A força bruta leva de cerca de 404 s a 17 h para 10^7 leituras. Dentro da ordenação, o mergesort exige um vetor auxiliar de 20 MB e o quicksort cerca de 740 bytes de pilha; só o heapsort, iterativo e in-place, cabe. Hash, AVL, Bloom e bitmap precisam de kilobytes.
- **Alternativas descartadas para o Datacenter:** hash, AVL e Bloom também cabem em 2 s, mas o hash não tem garantia de pior caso ($O(n^2)$), o AVL encosta nos 2 s na estimativa conservadora e o Bloom perde dados. O bitmap é exato, determinístico e o mais rápido.

A análise das dez estratégias (tempo, espaço, melhor/médio/pior caso, aceitação por cliente e rejeição numérica das quatro absurdas) está em *docs/analise_teorica.md*.

3 – Resultados medidos

Cada ponto é a mediana de 5 rodadas, sobre cópias novas de dados pseudoaleatórios com semente fixa. A memória é a soma do pico de heap (alocador com contador) e do pico de pilha (técnica da pintura, um limite superior). O vetor de entrada não conta. Máquina de medição: Intel Xeon 2,1 GHz (2 vCPU), Linux, GCC 13.3 com -O2; a máquina de referência da disciplina deve repetir a medição rodando o programa *main*.

n	Heapsort tempo mediano	Heapsort memória extra	Bitmap tempo mediano	Bitmap memória extra	Únicos
10^3	52,7 µs	71 B (heap 0 + pilha 71)	1,7 µs	8 479 B (heap 8 192 + pilha 287)	990
10^5	12,08 ms	71 B (heap 0 + pilha 71)	427,9 µs	8 479 B (heap 8 192 + pilha 287)	51 340
10^7	1,951 s	71 B (heap 0 + pilha 71)	12,34 ms	8 479 B (heap 8 192 + pilha 287)	65 536

Casos de teste obrigatórios:

- **Corretude:** casos com n ≤ 10, com e sem duplicatas, todas iguais, extremos 0 e 65 535, um elemento e vazio, nas duas estratégias: todos corretos.
- **Limite de memória da Embarca:** 71 bytes (heap 0 + pilha 71) em todos os n, abaixo de 256.
- **Orçamento do Datacenter:** n = 10^7 em 12,34 ms (mediana) e 12,66 ms (pior das 5 rodadas), abaixo de 2 s.

4 – Teoria × prática e gráficos com as dez estratégias

As curvas das oito estratégias não implementadas usam o número teórico de passos multiplicado por duas constantes medidas: K_1 = 1,23 ns por passo simples (bitmap, n = 10^7) e K_2 = 8,39 ns por $n \cdot \log_2 n$ (heapsort, n = 10^7). A tabela compara o tempo medido com o previsto.

n	Bitmap: medido ÷ previsto ($K_1 \cdot n$)	Heapsort: medido ÷ previsto ($K_2 \cdot n \cdot \log_2 n$)
10^3	1,41	0,63
10^5	3,47	0,87
10^7	1,00	1,00

Os tempos seguem a ordem de grandeza da teoria. As divergências foram explicadas e nenhuma muda os vereditos:

- O bitmap fica 3,5 vezes acima da reta em n = 10^5. A causa provável, que não verificamos com profiler, é a previsão de desvio do processador: nesse tamanho cerca de metade das leituras é nova (51 340 únicos em

10^5), então o *if* é imprevisível; em 10^3 quase todas são novas e em 10^7 quase todas são repetidas, e nesses casos o desvio é previsível.

- O heapsort é 6,8 vezes mais lento que o previsto com K_1, porque acessa posições distantes de um vetor de 20 MB (maior que o cache) e erra o cache muitas vezes; o bitmap mexe em 8 KB, que cabem no cache. Por isso as estratégias n log n usam K_2.
- A força bruta prevê 61 678 s ($n^2/2$), mas como só há 65 536 valores possíveis a lista de únicos para de crescer e o custo real fica em cerca de 404 s.
- Hash, AVL e Bloom têm espaço O(n) na teoria, mas o número de distintos nunca passa de 65 536; as curvas seguem a teoria geral do catálogo.
- Ambiente (cache, sistema operacional, relógio): em n = 10^3 o ruído fixo pesa (microssegundos), e em n = 10^7 o heapsort variou de 1,937 s a 2,079 s entre as rodadas.

Pontifícia Universidade Católica do Paraná – PUCPR
Escola Politécnica – Campus Sede – Curitiba/PR
Curso Bacharelado em Ciência da Computação
Disciplina de **Complexidade de Algoritmos**
Abílio Pedro Alcântara Mota Batista

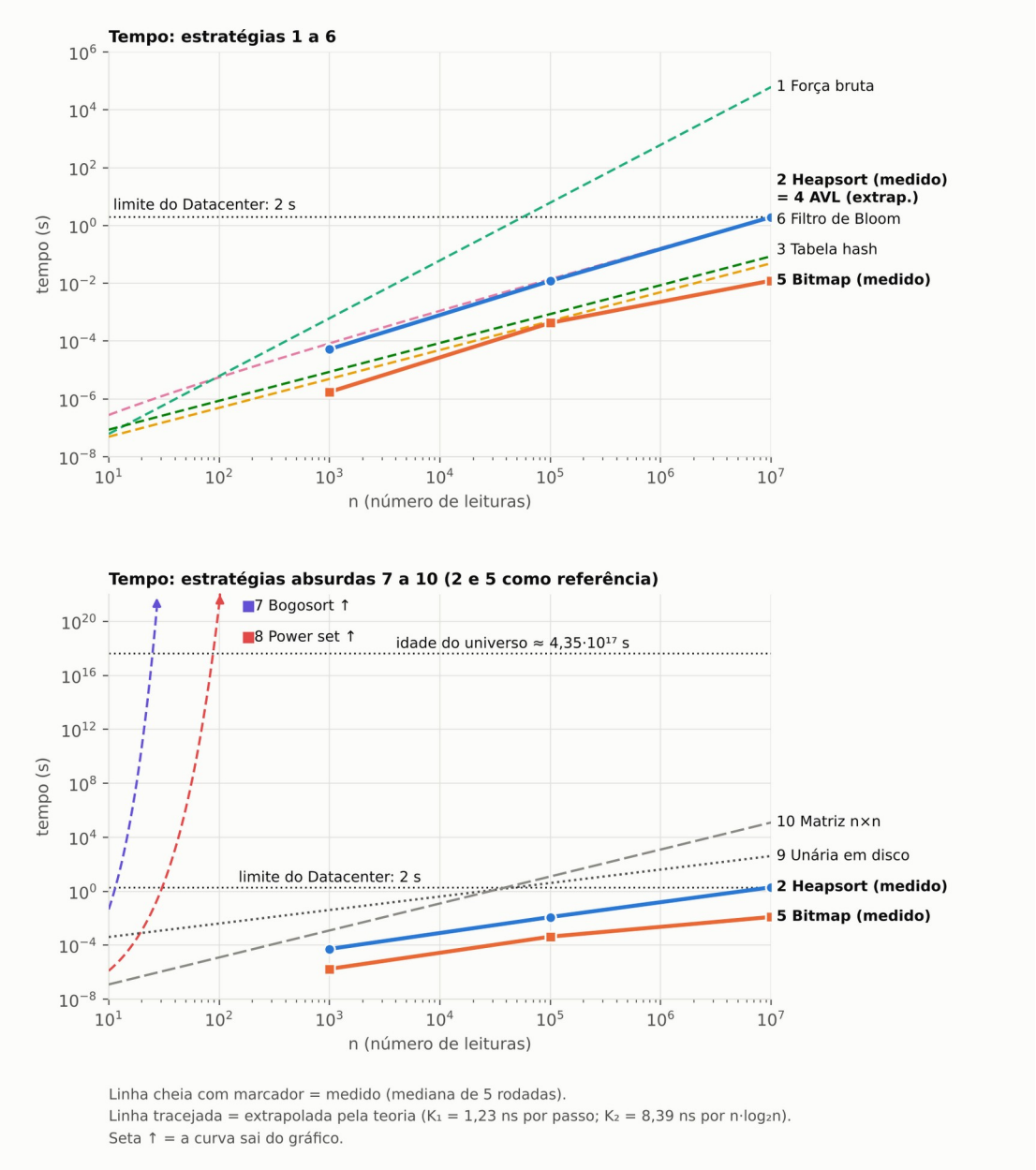


Figura 1. Tempo de execução × n (escalas logarítmicas). Linha cheia: medido. Tracejada: extrapolado pela teoria. Pontilhada: limite do Datacenter (2 s) e idade do universo.

Pontifícia Universidade Católica do Paraná – PUCPR
Escola Politécnica – Campus Sede – Curitiba/PR
Curso Bacharelado em Ciência da Computação
Disciplina de **Complexidade de Algoritmos**
Abílio Pedro Alcântara Mota Batista

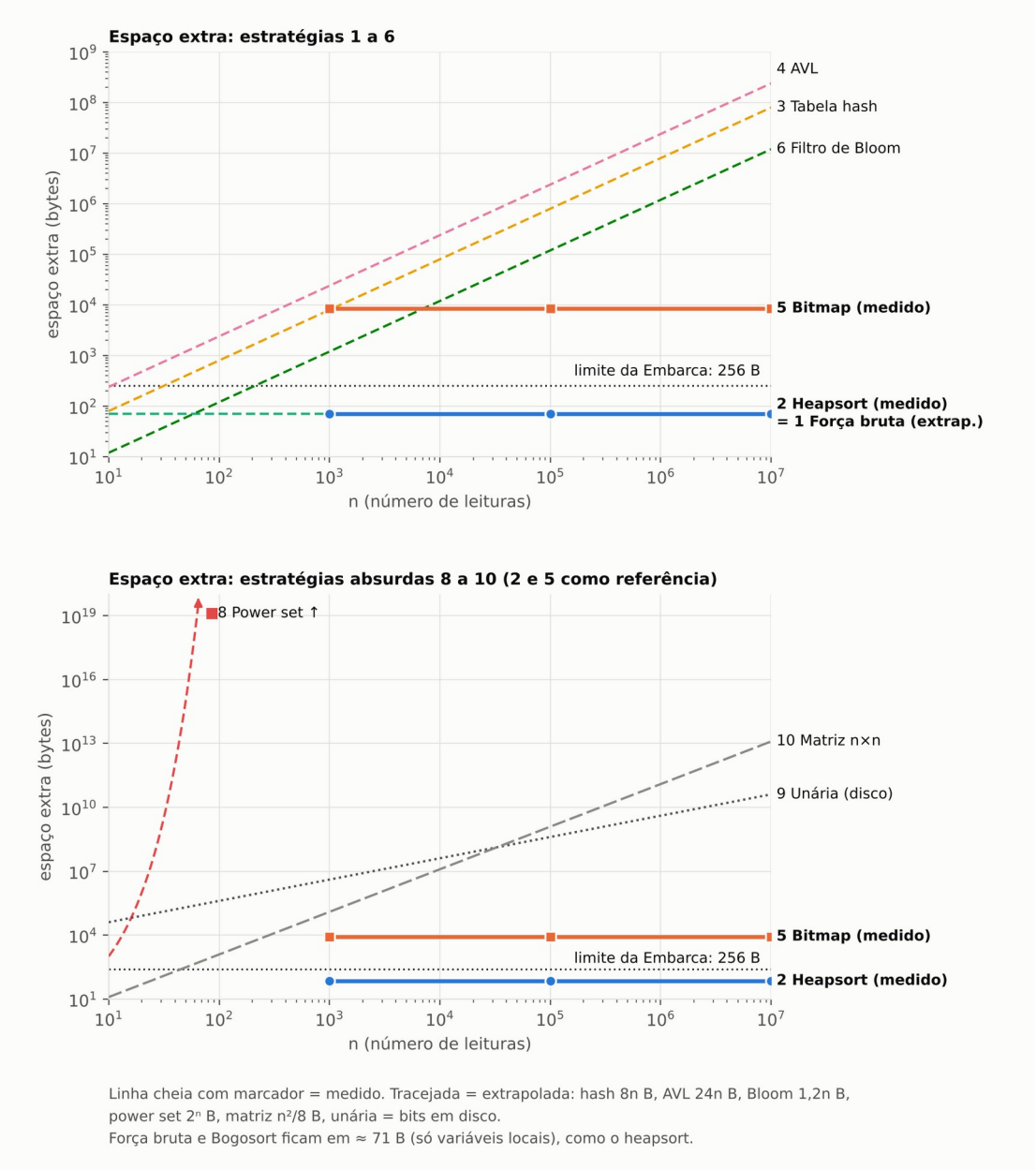


Figura 2. Espaço extra × n (escalas logarítmicas). Pontilhada: limite da Embarca (256 bytes). Força bruta e bogosort ficam em cerca de 71 bytes, como o heapsort.